대화형이며 강건한 메시 Boolean 연산

Interactive and Robust Mesh Booleans

Gianmarco Cherchi — University of Cagliari, Italy

Fabio Pellacini — Sapienza University of Rome, Italy

Marco Attene · Marco Livesu — CNR IMATI, Italy

번역 안내: 첨부 논문의 본문·알고리즘·표·그림 설명·감사의 글을 한국어로 번역했다. 원문 페이지는 삽입하지 않았으며, 그림과 수식은 원본을 유지했다. 참고문헌의 서지 정보는 검색과 대조를 위해 원문으로 보존했다. Boolean은 합집합·교집합·차집합을 포함하는 기하 연산을 뜻한다. arrangement는 교차 경계를 반영하여 일관되게 분할된 메시 구조라는 뜻으로 '메시 배열'로 옮겼다.

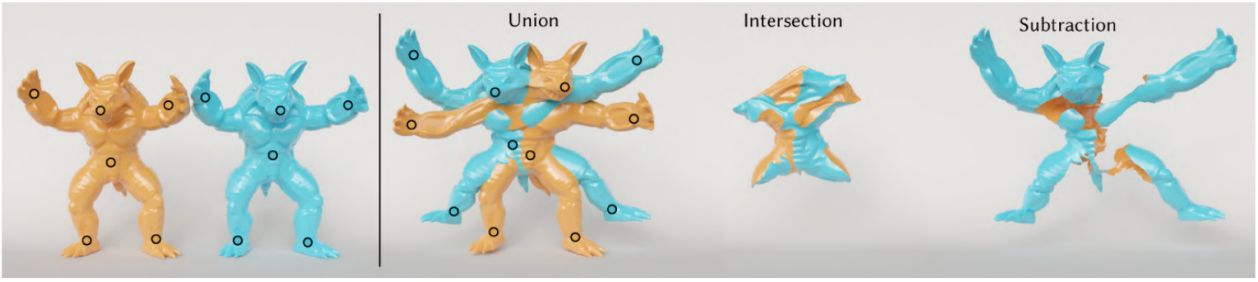


그림 1. 수천 삼각형의 단순하지 않은 메시에서 사용자가 강건한 Boolean을 실시간으로 수행할 수 있다. 먼저 임의 수의 변형 핸들을 선택한다(왼쪽). 핸들을 자유롭게 움직이면 시스템이 ARAP 변형[Sorkine와 Alexa 2007]과 강건한 Boolean을 실시간 적용한다. Armadillo 메시 두 개는 각각 삼각형 5만 개이다.

초록

Boolean 연산은 디지털 형상을 생성하고 편집할 때 가장 널리 쓰이는 방식 중 하나이다. 개념은 단순하지만 메시 Boolean의 계산은 매우 어렵다고 알려져 있다. 주요 문제는 수치 근사 때문에 교차점의 검출과 처리가 일관성과 신뢰성을 잃는 데서 발생한다. 그 결과 부동소수점 산술에 기반한 구현은 여러 퇴화 상황과 실패에 노출된다.

유리수 또는 정확한 기하 판정식에 기반한 수치 방법은 필요한 강건성 보장을 제공하지만 계산 시간이 증가한다. 지금까지 이 비용은 강건한 메시 Boolean의 사용을 오프라인 응용으로 제한해 왔다. 우리는 강건성을 보장하면서 최대 20만 개 삼각형의 메시에서 상호작용 가능한 프레임률로 작동하는 Boolean 알고리즘을 소개한다.

도구를 철저히 평가하며, 대화형 응용뿐 아니라 대규모 메시 집합의 일괄 처리, 수백만 요소를 포함하는 거대 메시 처리, 수백 형상을 한꺼번에 다루는 다항 Boolean 연산을 고려했다. 모든 실험에서 기존의 강건한 부동소수점 방법보다 일관되게 최소 한 자릿수 규모, 즉 약 10배 이상의 속도 향상을 얻었다.

CCS 분류: 컴퓨팅 방법론 → 메시 기하 모델. 추가 주제어: 계산 기하, 솔리드 모델링, CSG, 대화형 모델링, 메시.

1. 서론

Boolean 연산으로 3D 메시를 결합하는 것은 구성적 솔리드 기하(CSG)로 복잡한 형상을 정의하는 기본 기능이다. 사용자에게는 직관적이지만, 작은 수치 오차가 예측할 수 없는 위상 오류를 만들 수 있어 정확히 구현하기 어렵다. 대화형 모델링 소프트웨어는 상호작용성을 얻기 위해 주로 강건하지 않은 방법에 기반한다. 그러나 사용자 포럼의 실패 사례 모음이 보여주듯, 이로 인해 작업 과정에 상당한 문제가 생긴다. Autodesk Maya와 Blender 같은 전문 소프트웨어도 동일 평면성 같은 어려운 배치에서 불안정성이 발생함을 공개적으로 설명한다[Blender Doc 2022; Maya Doc 2022].

학계는 이 문제를 오랫동안 연구해 강건한 알고리즘을 개발하고, 때로는 [Jacobson 외 2018]처럼 무료 구현도 제공했다. 그러나 현재까지 강건한 방법은 모든 곳에서 정수 좌표를 요구하여[Trettner 외 2022] 다른 부동소수점 기하 처리 작업과 기본적으로 호환되지 않거나, 상호작용 가능한 프레임률을 확보하기에는 여전히 느리다. 따라서 공학·제작용 오프라인 모델링 등에 사용이 제한된다[Alderighi 외 2019, 2018; Attene 2018; Dai 외 2018; Fanni 외 2018; Garg 외 2016; Jacobson 2017; Muntoni 외 2018; Nuvoli 외 2019; Ureta 외 2016; Yao 외 2017].

많은 기존 방법은 메시 Boolean 계산을 두 단계로 구성한다. 먼저 교차선이 연결 구조에 포함되도록 삼각형을 나누어 메시 요소 사이의 충돌을 해소한다. 다음으로 각 요소가 입력 물체의 내부인지 외부인지 판정한다. Boolean 결과는 첫 단계에서 생성한 요소를 두 번째 단계의 내부·외부 표지에 따라 걸러낸 부분집합이다. 예를 들어 삼각형 메시 A와 B의 합집합 A∪B는 B 외부에 있는 A의 삼각형과 A 외부에 있는 B의 삼각형으로 구성된다(그림 2).

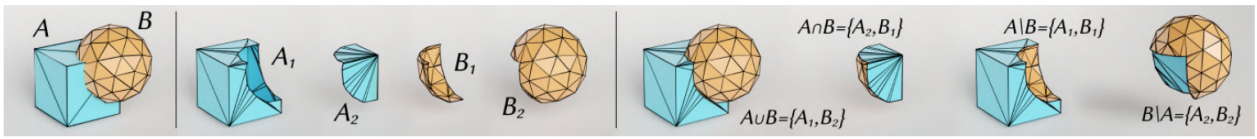


그림 2. 두 형상의 Boolean 계산은 (i) 교차를 해소하여 경계가 일치하는 표면 패치를 만들고, (ii) 선택한 Boolean 연산자에 따라 패치를 결합하여 결과를 생성하는 과정이다. 표기: Input=입력, Split=분할, Union=합집합, Intersection=교집합, Difference=차집합.

Boolean 파이프라인 구현의 가장 큰 어려움은 유한 정밀도 산술로 교차점을 정확히 표현하거나 검사할 수 없다는 것이다. 실제로는 불완전한 교차선부터 내부·외부 관계를 정의할 수 없게 만드는 위상적으로 일관되지 않은 분할까지 여러 결함을 만든다. 그림 3은 크게 겹친 두 원판의 차집합에서 심각한 문제가 생기는 전형적인 실패이다.

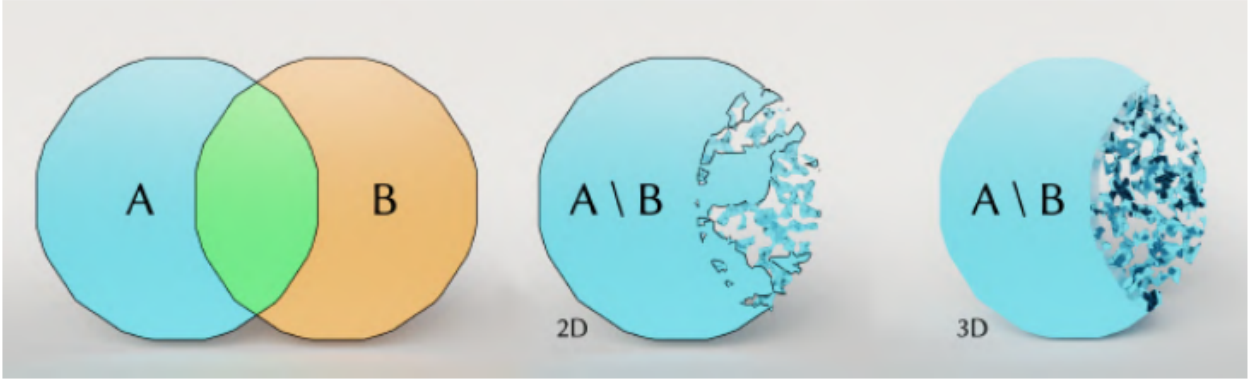


그림 3. Cork[Bernstein 2013]의 두 실패 사례. A B의 2D 버전(가운데) 오일러 지표는 −50이고 3D 버전(오른쪽)은 16이다. 두께가 있는 원판은 솔리드 물체만 지원하는 Geogram/Graphite[Levy 2022]의 실험 도구로도 시험했으며 프로그램이 비정상 종료했다.

강건한 내부·외부 분할 방법은 존재하지만 계산 효율이 낮거나[Jacobson 외 2013], 질의 시에는 효율적이어도 근사 방법이고 초기화가 필요하다[Barill 외 2018]. 대안은 부동소수점 산술을 피하고 점 좌표를 유리수[Zhou 외 2016] 또는 암시적 방식[Diazzi와 Attene 2021]으로 표현하는 것이다. 이 기법들은 위상적으로 올바른 결과를 보장하지만 부동소수점보다 10~100배 느리므로 대화형 사용에 적합하지 않다.

이 논문은 최신 정확 알고리즘의 효율을 최소 약 10배 개선하고 전처리 없이, 일반 메시의 대화형 응용에 정확한 부동소수점 방법의 강건성을 제공한다. 6절에서 보이듯 일반 노트북에서 최대 20만 삼각형을 포함하는 메시를 대화형으로 편집할 때 강건한 Boolean을 사용할 수 있다. 또한 대규모 데이터 집합의 일괄 처리, 수백만 삼각형의 제작용 고해상도 메시, 수백 입력 형상의 다항 Boolean 같은 실용적인 오프라인 응용에서도 기존 방법보다 훨씬 좋은 성능을 보인다.

두 파이프라인 단계 모두에 대한 기여로 개선이 가능해졌다. 첫 단계는 [Cherchi 외 2020]의 메시 배열에서 파생된 방법을 4절처럼 개선하여 평균 5배 이상 가속했다. 두 번째 단계는 배열의 위상적 정확성 보장을 활용하고 강건한 광선 투사와 결합하여 패치마다 광선 하나만 발사해 내부·외부 표지를 신뢰성 있게 계산한다.

강건한 판정식[Attene 2020; L vy 2016; Shewchuk 1997]에서 착안해, 내부·외부 검사를 빠르지만 강건하지 않은 방법부터 느리지만 강건한 방법까지 순차적으로 적용하는 광선 투사 단계들로 구성한다. 완전히 정확하고 강건한 광선 투사는 꼭 필요한 경우에만 사용한다. 5절에서 설명하듯, 이 방법은 기존 정확 방법의 위상적 전파[Attene 2014]나 패치 그래프 처리[Zhou 외 2016]보다 최대 100배 빠르다. 수백만 삼각형의 큰 메시와 수백 메시의 다항 Boolean에도 최적으로 확장된다(6절).

우리는 이 연구가 수치 계산 속도 측면에서 최신 기술을 상당히 개선하고, 더 나아가 대화형 응용에 강건성을 제공한다고 생각한다. 실험은 강건한 Boolean을 다른 기하 처리와 실시간으로 결합하는 방법을 탐색한다. 이는 향후 대화형 응용의 가능성 중 일부에 불과하다. 실험과 향후 연구를 지원하기 위해 프로토타입을 오픈소스로 공개한다.

2. 기존 기술

Boolean 조합은 결과의 명시적인 3D 모델을 계산하지 않고도 표시할 수 있다. 시각화 및 몇몇 특정 응용에는 이것으로 충분하다[Baxter와 Wrigh 2019; Chen 외 2022; Zanni 외 2018]. 그러나 일반적인 모델링 시스템에는 Boolean 조합의 명시적 표현이 필요하고, 대상 응용에 따라 계산이 강건하고 정확해야 한다.

기존 알고리즘은 수치 모델(부동소수점 또는 정확 산술), 기하 접근법(표면 또는 체적 기반), 결과 유형(정확 또는 근사)에 따라 분류할 수 있다.

2.1. 수치 모델

Boolean을 구현하는 가장 쉬운 방법은 부동소수점 산술이다. 하드웨어 지원 때문에 부동소수점은 가장 빠르며, 학술 구현[Levy 2022]과 Autodesk Maya·Blender 같은 전문 구현 모두에서 널리 사용된다. 그러나 서론에서 언급했듯 반올림 오차 때문에 매우 취약하여 그림 3과 같은 심각한 위상 오류를 일으킬 수 있다.

반대 극단에서는 부동소수점을 유리수 같은 정확한 수치형으로 대체하여 무조건적인 강건성을 얻을 수 있다[Schifko 외 2010]. 그러나 정확 산술은 부동소수점보다 10배 이상 느려 허용하기 어려운 성능 저하를 만든다.

어떤 기하 처리에서는 반올림 오차와 무관하게 프로그램 흐름만 정확하게 보장해도 강건성을 얻을 수 있다. 산술 필터링으로 기하 판정식을 빠르고 정확하게 평가하는 방식이다[L vy 2016; Shewchuk 1997]. 전형적인 판정식은 다항식의 부호를 계산하며, 부호가 정확하면 프로그램 흐름도 일관되게 유지된다.

산술 필터링[Devillers와 Preparata 1998]은 부동소수점으로 다항식을 평가하면서 반올림 오차의 상한도 계산한다. 계산된 식의 크기가 오차 상한보다 크면 부호가 정확함을 보장한다. 그렇지 않으면 필터가 실패하고 임의 정밀도로 판정식을 재평가한다. 실패율이 충분히 낮으면 절대 정밀도가 드물게 사용되므로 성능 저하가 허용 가능하다[Magalh es 외 2017]. 특히 병렬 아키텍처에서 그렇다[de Magalh es 외 2020].

산술 필터링은 판정식 입력의 전역적 일관성이 보장될 때만 사용할 수 있다. 따라서 입력점 좌표에 직접 작동하는 메시 생성처럼 판정식의 입력이 참값에서 오는 작업에만 사용된다[Hang 2015; Shewchuk 1996]. 중간 기하 구성을 사용하는 작업에서는 지연 정확 평가가 현재의 표준 해법이지만[Pion과 Fabri 2011], 대화형 응용에는 여전히 느리다.

여러 연구가 관찰했듯 메시 Boolean의 중간 구성은 교차점뿐이다[Bernstein과 Fussell 2009; Campen과 Kobbelt 2010a,b; Sugihara 외 1989]. 이 점들을 입력 기본 요소의 교차로 암시적으로 표현하고, 점의 식과 판정식의 식을 합성하면 산술 필터링을 사용할 수 있다[Attene 2020]. 우리 방법은 이 기법으로 입력점과 교차점의 임의 조합에 대한 기하 질의를 빠르고 정확하게 수행한다.

2.2. 표면 및 체적 기반 방법

Boolean의 한 단계 또는 두 단계 모두를 구현하는 방법은 명시적인 체적 메시나 관심 체적을 둘러싼 표면 메시로 작동할 수 있다. 일반적으로 명시적인 체적 표현은 내부·외부 표지 같은 작업을 쉽게 하지만 차원이 증가해 효율이 낮다. 표면 기반 방법은 조금 복잡하지만 대체로 성능이 좋다. 우리 방법은 후자에 속한다.

체적 기반 방법. 초기 Boolean 연구는 주로 구·원기둥·반공간 같은 단순 체적 기본 요소 또는 일반 입력을 위한 암시적 표현에 기반했다[Pasko 외 1999; Wyvill 외 1999]. [Sell n 외 2021] 같은 예외도 있지만, 현대 기하 모델링의 형상 복잡도가 증가하면서 명시적인 메시 표현이 점차 널리 쓰였다.

[Hu 외 2018]은 CGAL의 정확 커널을 사용해 입력 메시로 공간을 일치하는 체적 셀로 나누어 메시 배열을 얻는다. 더 빠른 버전[Hu 외 2020]은 부동소수점 계산으로 일치하는 사면체 메시를 생성하지만 형식적인 보장은 없다. [Diazzi와 Attene 2021]도 입력과 일치하는 체적 메시를 만들며, 정확 산술 대신 간접 판정식을 사용하여 정확성 보장을 유지하면서 Boolean을 훨씬 빠르게 추출한다.

[Paoluzzi 외 2020, 2019]는 부동소수점 계산과 기하·대수 위상학의 언어로 입력 면에 따른 공간 분할을 다룬다. [Tao 외 2019]는 축 정렬 평면으로 입력 삼각형을 자르고 체적 셀을 정의해 부동소수점의 취약성을 줄였다. 그러나 순수 부동소수점 기반이므로 정확성 보장은 없다.

표면 기반 방법. [Schifko 외 2010]은 입력 표면을 따라 이동하며 교차를 만나면 삼각형을 분할하는 정확한 기하 구성을 사용한다. 그 비용이 크므로 [Attene 2014]는 외부 표면을 순회하되 반올림이 해롭지 않을 때는 부동소수점 구성, 필요할 때는 정확 산술을 사용하는 하이브리드 접근법을 제안한다. [Xu와 Keyser 2013]는 정확한 기하 구성 없이 방향 판정식을 영리하게 사용해 결과 표면의 위상적 정확성을 보장한다.

[Mei와 Tipper 2013]은 표면을 순회하는 대신 임시 옥트리로 교차 후보 삼각형 쌍을 빠르게 찾는다. [Zhou 외 2016]은 CGAL 정확 커널로 공간을 셀로 나누고 각 셀에 입력에 대한 감김수를 부여한다. 표지에 따라 셀 부분집합을 선택해 Boolean을 수행한다.

[Attene 2020]의 교차점·판정식 대수 합성은 [Cherchi 외 2020]에서 임의로 자기 교차하는 삼각형 집합을 올바른 단체 복합체로 빠르게 변환하는 데 사용되었다. 가장 빠른 배열 알고리즘임에도 원래 구현은 대화형 응용에 충분히 빠르지 않다. 우리 첫 단계는 여기에 크게 기반하지만 모듈 일부를 교체하고 자료구조와 실행 흐름을 병렬화에 적합하게 재설계하여(4절) 평균 5배 가속했다(6절).

2.3. 정확 및 근사 방법

근사를 허용하는 응용에서는 입력 정점 좌표를 삼각형 평면 계수로 변환한 뒤 쉽게 결합해 Boolean을 구현할 수 있다[Bernstein과 Fussell 2009]. 변환은 정확하지 않아 보통 결과를 복구해야 한다. [Barill 외 2018], [Hu 외 2020]은 강건성과 속도를 결합한 또 다른 근사 Boolean 방식이다. [Pavi 외 2010]은 교차선 부근을 정확히 자르는 대신 촘촘하게 재메시한다.

그러나 근사를 허용할 수 없는 경우도 있다. 설계자가 수많은 연속적이고 예측 불가능한 작업을 수행하는 대화형 모델링에서는 근사가 빠르게 누적되어 나쁜 결과를 만들 수 있다. 이 경우 느린 임의 정밀도[Schifko 외 2010; Zhou 외 2016] 또는 부동소수점 반올림의 영리한 처리를 통해 정확한 결과를 얻을 수 있다.

[Campen과 Kobbelt 2010a]는 입력 모서리를 충분히 짧게 나누어 [Bernstein과 Fussell 2009]의 좌표-평면 변환이 무손실이 되도록 보장한다. 이후 복구 없이 Boolean 조합과 다른 모델링 연산[Campen과 Kobbelt 2010b]을 수행할 수 있다. 그러나 연산이 연속 적용되면 평면 기반 결과가 매우 복잡해진다. [Sheng 외 2018]은 변환을 줄이기 위해 좌표와 평면을 모두 사용한다. [Nehring-Wirxel 외 2021]은 중간 평면 표현을 단순화하는 효과적인 방법을 보고했다.

최근에는 메시 배열[Cherchi 외 2020]과 다면체 공간 분할[Diazzi와 Attene 2021]에서 중간 기하 구성 대신 간접 판정식을 사용하는 흐름이 있다. Boolean 계산에 사용하면 속도를 희생하지 않고 강건성을 보장한다. 다만 최종 응용에 명시적인 체적이 필요 없다면 이를 계산하는 것은 불필요한 오버헤드이다.

스냅 반올림. 정확 방법은 유용하지만 결과를 파일에 저장하거나 정확하지 않은 다른 알고리즘으로 전달할 때 공통 문제가 있다. 정확하거나 암시적인 좌표를 부정확한 부동소수점 값으로 바꿔야 하고, 이 근사가 퇴화 또는 교차 요소를 만들어 모델을 무효화할 수 있다. 정확성이 증명되고 효율적인 해법은 아직 없으며, 기존 방법은 비실용적이거나[Devillers 외 2018; Fortune 1999] 모든 경우의 정확성을 보장하지 못한다[Milenkovic과 Sacks 2019]. 그러나 기존 휴리스틱은 실제 사례에서 매우 작은 비율로만 실패했다[Zhou 외 2016].

정확 방법으로 만든 메시는 근사 방법으로 얻지 못하는 위상적 속성을 가진다. 예를 들어 접선 방향으로 접촉하지 않는 수밀한 다양체 메시 두 개의 Boolean 결과는 수밀한 다양체임이 보장된다. 스냅 반올림으로 기하 퇴화가 생기더라도 이런 속성은 후속 응용에 중요할 수 있다.

2.4. 내부·외부 분류

대부분의 메시 Boolean 알고리즘은 각 요소가 결과에 포함되는지 판단해야 한다. 표면 기반에서는 삼각형이 결과의 경계를 이루는지 알아야 한다. 체적 기반에서는 셀이 체적 결과에 포함되는지 알아야 한다.

표면 기반의 일반적인 방식은 표면을 순회하고 기하학적 추론으로 결과에 속한 부분을 추적하는 것이다 [Attene 2014; Schifko 외 2010]. 정확 방법은 입력 배열이 올바름을 보장하므로 체적 분할의 셀을 무한한 외부 셀에서 시작해 쉽게 분류할 수 있다. 입력 표면을 지날 때 외부에서 내부로, 또는 반대로 바꾼다. 단순한 부동소수점 구현은 위상적 보장이 없어 이 방법과 호환되지 않는다.

표면에 구멍이 있거나 모호한 자기 교차가 있으면 일반화 감김수가 효과적이다[Jacobson 외 2013]. 경우에 따라 그래프 컷 기반 알고리즘이 더 좋은 해법을 제공한다[Diazzi와 Attene 2021]. 감김수의 단순 계산은 느려 [Barill 외 2018]에서 빠른 근사 알고리즘을 제안했다. 이들은 일반적으로 위상 순회보다 느리지만, 단순 부동소수점에 기반한 부정확 방법에서는 피할 수 없는 비용이다.

우리 분류 시스템은 정확한 광선 투사에 기반하며(5절), 두 계열보다 훨씬 빠르고(6절), 수백 입력 형상의 복잡한 다항 Boolean에도 더 잘 확장된다(6.4절).

최근 EMBER라는 강건한 대화형 Boolean 방법이 제시되었다[Trettner 외 2022]. 동차 정수로 점 좌표를 정확히 표현한다. 적극적인 병렬화 덕분에 수백만 요소의 메시에서도 대화형으로 작동하는 가장 빠른 기존 방법이다.

반면 정수 좌표를 선택했기 때문에 기존 기하 처리와 기본적으로 호환되지 않는다. Boolean을 부동소수점 작업과 연결하면 대화형 응용에서 매 프레임 손실 있는 입력 좌표 변환이 필요하다. 또한 병렬화 모델이 입력 메시를 작은 조각으로 나눠야 하므로 교차선 외에 불필요한 이음선이 생겨, 메시의 일치성을 활용하는 [Nuvoli 외 2019] 같은 알고리즘의 결과를 해칠 수 있다.

EMBER와 우리 시스템은 빠르고 강건한 Boolean을 서로 다른 방향에서 다룬다고 본다. EMBER는 입력 메시를 변경하는 대신 속도를 우선한다. 우리는 실행이 더 느릴 수 있더라도 입력을 유지해 기존 기하 처리 알고리즘과의 호환성을 우선한다.

3. 개요

입력은 메시 집합 M1, M2, …, Mn과 합집합·교집합·차집합 중 하나의 Boolean 연산자이다. 입력 메시가 모호함 없이 체적을 둘러싼다고 가정한다. 즉 다양체이며, 수밀하고, 자기 교차가 없어야 한다. 출력 메시 B는 입력에 연산자를 적용한 결과를 담는다.

Boolean 정의에 따라 B는 입력 체적의 부분체적이고 입력 삼각형의 일부가 경계를 이룬다. 결과와 입력 형상의 대응은 여러 응용에서 유용하므로 출력 삼각형마다 출처 정보를 전달한다. 동일 평면 영역에서 겹치는 메시처럼 출력 삼각형 하나가 여러 입력 메시에 속할 수도 있다.

서론처럼 알고리즘은 두 단계이다. 먼저 교차 메시 요소를 분할하여 연결 구조에 교차선을 포함한다. 정확한 방법을 사용하면 올바른 단체 복합체가 생성되며, 교차선인 비다양체 모서리의 닫힌 루프가 표면 패치의 경계를 이룬다.

두 번째 단계에서는 이를 이용해 각 표면 패치가 각 입력 메시의 내부인지 외부인지 결정한다. 이 정보로 패치를 걸러 결과를 얻는다. 예를 들어 M1∪M2는 M2 외부의 M1 패치와 M1 외부의 M2 패치이다. 다음 두 절에서 각 단계의 기술적 기여를 설명한다. 지원하는 모든 연산자의 시각적 예는 그림 2에 있다.

4. 교차 해소

이 모듈에서는 입력 메시를 교차 가능성이 있는 삼각형 집합으로 본다. 모든 입력 삼각형을 하나의 배열로 평탄화하고 각 삼각형이 속한 입력 메시의 태그를 연결한다.

높은 수준에서 기존 교차 해소 알고리즘은 모두 비슷하다. 먼저 삼각형 교차를 검출하고, 메시를 세분화하면서 교차점부터 삽입한 다음 선분을 삽입한다[Attene 2014; Cherchi 외 2020; Zhou 외 2016](그림 4). 차이는 교차점의 표현과 처리, 선분 삽입 방식 같은 기술적 세부 사항이다. 이는 속도, 메모리 효율, 병렬화, 대규모 데이터 확장을 보장하는 데 핵심이다.

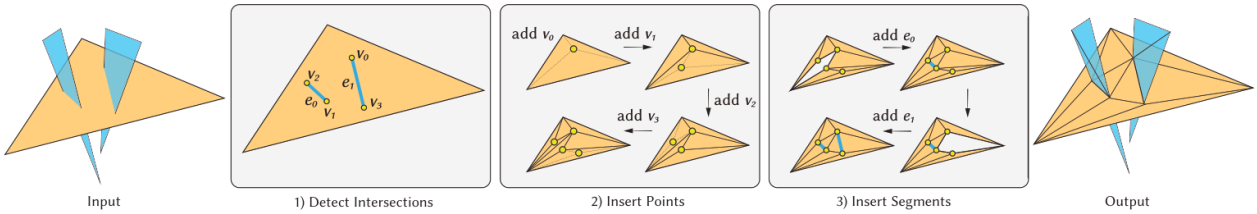


그림 4. Boolean 파이프라인의 첫 단계. 일반적인 삼각형 집합(왼쪽)에서 교차점과 교차선을 검출한다(1). 새 점을 먼저 포함하도록 삼각형을 분할하고(2), 다음으로 선분을 포함한다(3). 결과는 올바른 단체 복합체이다(오른쪽). 표기: Detect Intersections=교차 검출, Insert Points=점 삽입, Insert Segments=선분 삽입.

분할 단계는 [Cherchi 외 2020]에 기반한다. 해당 계열에서 가장 빠르지만 목표 메시 크기의 대화형 사용에는 충분하지 않아, 교차 검출과 교차선 삽입(그림 4의 1·3단계)을 개선했다. 전체적으로 원래 알고리즘보다 3~8배 빨라졌다. 아래에 주요 개선을 설명하며 전체 알고리즘의 자세한 논의는 원 논문을 참조한다.

캐시된 판정식. 무조건적인 수치적 강건성을 얻으려면 교차 검출, 정점 삽입을 위한 삼각형 내 점 위치 판정, 선분 삽입의 재삼각분할(그림 4의 1·2·3단계)이 모두 정확한 방향 판정식[Shewchuk 1997]에 기반해야 한다. 따라서 판정식은 분할 알고리즘의 계산 병목이다.

가장 빈번한 연산은 주어진 평면에 대해 공간의 점 위치를 판정하는 orient3D이다. 점 p와 a,b,c를 지나는 평면이 주어지면 다음 행렬식의 부호를 계산한다.

$$orient3D(a,b,c,p) = \begin{vmatrix} a_x & a_y & a_z & 1 \\ b_x & b_y & b_z & 1 \\ c_x & c_y & c_z & 1 \\ p_x & p_y & p_z & 1 \end{vmatrix}$$

전통적인 형태에서는 판정식을 호출할 때마다 행렬식을 처음부터 평가한다. 그러나 우리 알고리즘의 평면은 입력 삼각형의 지지 평면이므로 미리 알려져 있고, 여러 점에 반복적으로 검사된다. 따라서 행렬식 중 평면 부분을 한 번 계산하고 같은 평면에 새 점을 검사할 때 재사용하는 것이 유리하다. 이 생각으로 4×4 행렬식을 다음처럼 다시 쓴다.

$$
\begin{aligned}
orient3D(a,b,c,p) = &- p_x \begin{vmatrix} a_y & a_z & 1 \\ b_y & b_z & 1 \\ c_y & c_z & 1 \end{vmatrix} + p_y \begin{vmatrix} a_x & a_z & 1 \\ b_x & b_z & 1 \\ c_x & c_z & 1 \end{vmatrix} \\
&- p_z \begin{vmatrix} a_x & a_y & 1 \\ b_x & b_y & 1 \\ c_x & c_y & 1 \end{vmatrix} + \begin{vmatrix} a_x & a_y & a_z \\ b_x & b_y & b_z \\ c_x & c_y & c_z \end{vmatrix}
\end{aligned}
$$

이렇게 평면 계수와 점 좌표를 완전히 분리한다. 각 입력 삼각형에서 4개의 3×3 행렬식을 캐시하여 orient3D 호출을 간단한 4D 내적으로 줄인다. 비슷한 캐싱은 수십억 점의 대규모 점군 사면체화[Marot 외 2019] 및 평면 표현 기반 Boolean[Nehring-Wirxel 외 2021]에도 사용된다.

선분 삽입. 출력 메시가 교차선을 올바르게 포함하려면 교차 정점뿐 아니라 교차 선분도 삽입해야 한다(그림 4의 3단계). 현재 삼각분할에서 선분과 충돌하는 삼각형을 모두 제거하고, 원하는 선분이 새 삼각분할에 포함되도록 생성된 다각형 영역을 다시 삼각분할한다. 이는 계산 기하에서 오래전부터 널리 연구한 문제이다[Shewchuk과 Brown 2015].

[Cherchi 외 2020]은 earcut 알고리즘을 사용했다. 최상의 구현에서도 다각형 선분 수 n에 대해 최악의 복잡도는 $O(n^2)$이다[Eberly 2008]. 우리는 이를 [Livesu 외 2021]의 방법으로 대체했다. 모든 경우 최적인 결정론적 $O(n)$ 복잡도를 보장하고, 기존의 가장 좋은 방법[Shewchuk과 Brown 2015]보다 약 100배 빠르다.

저수준 구현. 실행 속도를 더 높이기 위해 기반 자료구조를 재설계하여 병렬 실행 가능성을 늘렸다. 가장 비싼 연산은 중복·퇴화 요소 제거, 인접성 구성, 교차 계산, 최종 삼각분할이다. 핵심 구성요소를 데이터 병렬로 실행해 파이프라인은 유지하면서 큰 속도 향상을 얻었다.

중첩 병렬 구문으로 옥트리 생성도 최적화했다. 병렬 구문은 작업 훔치기 스케줄러가 실행하여 작업량의 균형을 맞춘다. 어떤 모델링 작업도 사용자가 수행할 수 있게 하면서 강건성을 보장하기 위해 퇴화 제거와 인접성 재구축을 매 프레임 수행한다.

패치 구성과 최종 메시 추출에서도 추가 병렬화를 검토했다. 그러나 세밀한 잠금이 필요해 병렬성을 추출하기 어렵다. 실험했지만 속도 향상이 없었으므로 파이프라인의 추가 병렬화에는 meshlet 기반 접근이 필요할 가능성이 있다[Mahmoud 외 2021].

마지막 개선은 특수 자료구조를 사용해 캐시 일관성을 개선하고, 삭제로 인한 메모리 단편화와 시스템 메모리 할당기 부담을 줄이는 것이다. 메시 저장 방식이나 크기와 무관하게 자료구조 갱신에는 삭제를 포함한 수많은 작은 메모리 연산이 필요하고 성능 영향이 크다. 분할 단계의 가장 비싼 저수준 구조는 집합, 딕셔너리, 인접 리스트로 저장한 희소 그래프이다.

세 가지 기법으로 최적화한다. 첫째, 집합과 딕셔너리에 Swiss table 설계의 해시 테이블을 사용해 메모리를 절약하고 캐시 일관성을 높인다. 둘째, arena 할당기로 할당기 부담과 전체 단편화를 줄인다. 셋째, 인접 리스트에 작은 배열 최적화가 적용된 동적 배열을 사용한다. 이 기법들은 파이프라인 전반에 유의미한 속도 향상을 제공한다.

5. 내부·외부 분류

두 번째 단계에서는 앞서 계산한 단체 복합체의 각 다양체 표면 패치가 입력 메시 M1,⋯,Mn에 대해 어디에 있는지 결정한다. 기존 기법을 개선한 첫 단계와 달리, 이 분류는 기존 위상 기반 접근[Attene 2014; Zhou 외 2016]과 완전히 다르다.

핵심은 패치의 임의 점에서 임의 방향으로 광선을 발사하고 입력 메시 M과의 교차를 분석하면 패치와 M의 내부·외부 관계를 결정할 수 있다는 것이다(그림 5 오른쪽). 알고리즘은 삼각형 수가 아닌 다양체 패치 수에 따라 확장된다. 패치 수는 보통 삼각형보다 몇 자릿수 적고, 광선 하나의 비용은 거의 무시할 수 있어 매우 빠르다. 실험에서는 기존 방법보다 최대 100배 가속했고 여러 입력 형상의 다항 Boolean에도 더 잘 확장되었다(6절).

알고리즘 1. 내부·외부 분류

입력: 메시 M1,⋯,Mn과 분할된 패치 P1,⋯,Pm.

출력: 각 패치의 입력 메시 M1,⋯,Mn에 대한 상대 위치.

각 패치 P에 대해:

P를 M1,⋯,Mn의 외부로 초기화한다.

p∈P에서 시작하여 무한 원점 p∞로 향하는 광선 r을 정의한다. (5.1절)

각 입력 메시 M에 대해:

r과 M의 교차를 계산하고 정렬한다. (5.2절)

r과 M이 교차하면:

교차하는 삼각형 t∈M을 찾는다. (5.3절)

사면체 (t,p∞)의 체적을 계산한다. (그림 5)

체적이 음수이면 P를 M 내부로 설정한다.

조건문 종료.

입력 메시 반복 종료.

패치 반복 종료.

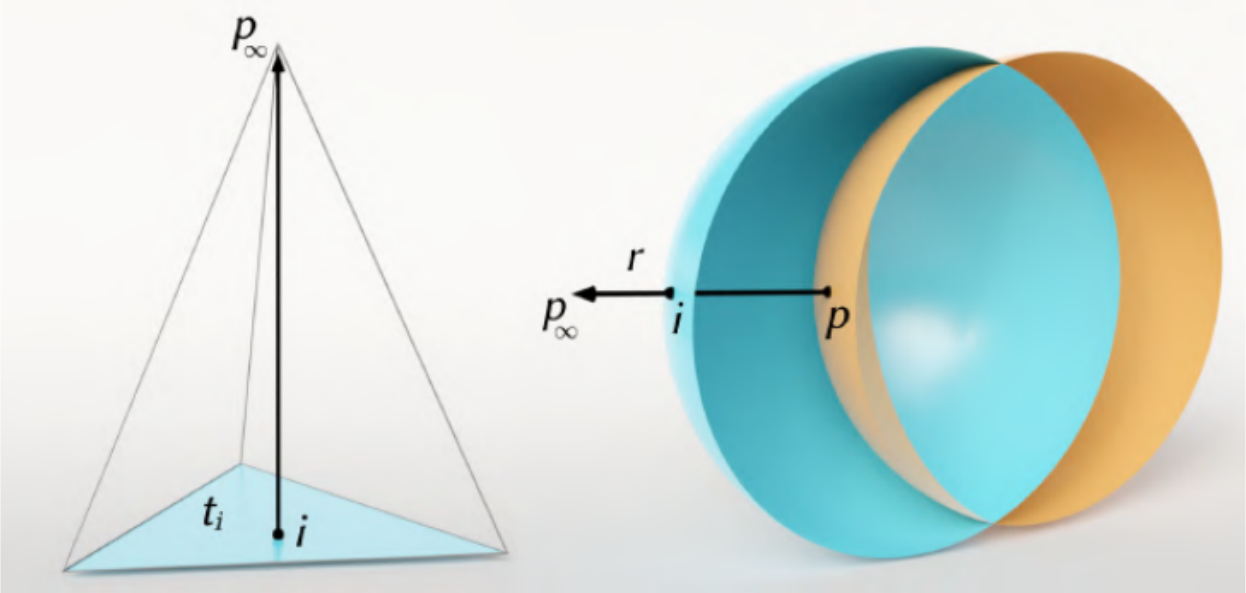


그림 5. 단체 복합체의 각 다양체 패치에서 정확한 광선 투사로 입력 형상과의 내부·외부 관계를 강건하게 판단한다. 무한대를 향해 광선을 발사하고 모든 입력 메시와의 첫 교차를 분석한다. 교차점 i와 이를 포함하는 삼각형 ti가 주어지면, 사면체 (ti,p∞)의 체적이 음수일 때 광선은 내부에서 외부로, 그렇지 않으면 외부에서 내부로 이동한다. 산술 방향 판정식으로 이 검사를 정확히 수행할 수 있다.

알고리즘 1은 광선 투사 방식의 주요 단계를 요약한다. 각 패치 P에서 출발하여 모든 입력 메시 외부에 있도록 보장된 무한 원점 p∞를 향하는 광선 r을 구성한다. 각 입력 M과 교차를 검사하고, 하나라도 발생하면 출발점에 가장 가까운 첫 교차를 선택한다.

광선이 내부에서 외부로 가는지 반대인지 결정하려면 광선 방향과 바깥쪽 국소 표면 법선을 비교한다. 그림 5 왼쪽처럼 이는 사면체의 부호 있는 체적 평가이며 방향 판정식으로 정확하게 계산할 수 있다.

여러 기술적 과제가 있다. 첫째, 강건한 판정식이나 유리수로 계산한 정확한 배열에만 적용할 수 있다. 강건하지 않은 기법은 서로 다른 패치를 연결하는 틈이나 작은 위상 통로가 없음을 보장하지 못한다. 둘째, 근사가 인공 교차를 만들거나 실제 교차를 놓칠 수 있으므로 교차 검출도 정확해야 한다. 셋째, 광선과 표면이 접할 때의 모호함을 적절히 처리해야 한다.

단체 복합체 생성 중 삽입한 교차점은 명시적 좌표가 알려져 있지 않아, 계산이 암시적 점 표현과 완전히 호환되어야 한다. 이 과정은 본질적으로 불안정해 절대 정밀도를 요구한다. 아주 작은 기하·위상 결함으로 교차 하나를 놓친 광선이 임의로 큰 패치 전체를 잘못 분류할 수 있다. 나머지 절에서 각 문제를 해결하는 단계를 설명한다.

5.1. 광선 정의

패치 P의 점 p∈P에서 시작해 무한 원점 p∞를 지나는 광선을 정의해야 한다. p가 알려져 있다면 주요 축 하나를 따라 해당 축의 입력 장면 바운딩 박스 범위보다 큰 거리만큼 p를 이동해 p∞를 정의할 수 있다. 모든 입력 외부에 있음이 보장된다. 축 정렬 광선은 좌표 하나를 버리고 3D 대신 2D 평면에서 계산할 수 있어 이후 광선-삼각형 분석을 크게 단순화한다.

주요 어려움은 출발점 p의 정의이다. 가장 단순하게는 패치 내부에 참값 부동소수점 좌표를 아는 입력 정점이 하나 이상 있으면 이를 사용할 수 있다. 그러나 단체 복합체에는 입력 정점이 없는 패치나 내부 정점 자체가 없는 패치도 있다. 이런 경우 명시적인 부동소수점 좌표를 정의하는 것이 본질적으로 불가능할 수 있다. 반올림으

로 광선 r이 패치 P에서 떨어져 분류 오류를 일으킬 위험이 있기 때문이다.

비싼 유리수 좌표로 바꾸면 수치 문제를 피할 수 있지만 크게 느려진다. 대신 필터 판정식의 단계적 접근 [Attene 2020; L vy 2016; Shewchuk 1997]에서 영감을 얻어 먼저 충분한 근사 부동소수점 해법을 찾고, 정확성이 보장되는 유리수는 예비 전략으로만 사용한다.

핵심 아이디어는 P 아래에서 시작해 P 내부의 점 p를 반드시 통과하는 광선을 정의할 수 있다면, 모든 교차를 정렬하고 p까지의 교차를 건너뛴 다음 p 이후 첫 교차로 분류하면 된다는 것이다.

전략은 다음과 같다. P에서 임의 삼각형 t를 선택하고 암시적 정점 좌표를 명시적 부동소수점으로 변환하여 근사 무게중심 bt를 계산한다. bt가 t 아래에 있도록 p∞ 정의에 사용한 축의 뒤쪽으로 이동한다. 삼각형 정점의 스냅 반올림이 성공하면 bt에서 출발해 p∞를 지나는 광선은 t와 P를 교차하므로 분류에 사용할 수 있다.

그러나 부동소수점 변환이 광선을 이동시킬 수 있어 이 교차가 존재한다는 보장은 없다. 축 정렬 덕분에 t와 bt를 광선 방향으로 직교 투영한 간단한 2D 삼각형 내 점 검사로 원하는 교차의 존재를 효율적이고 신뢰성 있게 확인할 수 있다. 실패하면 P의 다른 삼각형으로 같은 구성을 시도한다. 유효한 삼각형을 찾지 못하면 유리수로 정확한 무게중심을 계산하고 정확한 광선 투사를 수행한다.

실제로 입력 정점을 포함하지 않는 패치는 드물고 근사 전략의 실패는 더 드물다. 대규모 벤치마크(6.2절)에서 Boolean 약 3800회와 광선 8만 개 이상을 검사했다. 광선 중 2%만 스냅 반올림이 필요했다. 이 경우의 97%에서는 첫 삼각형으로 유효한 광선을 정의했고, 2%에서는 두 삼각형, 0.56%에서는 세 삼각형을 검사했다. 최악에도 패치당 5개였으며 유리수 기반 최종 단계는 한 번도 사용하지 않았다.

5.2. 교차 검출

일반적인 광선 투사에서는 효율적인 M ller-Trumbore 알고리즘[1997]으로 광선-삼각형 교차를 계산한다. 그러나 부동소수점 연산이 오차를 누적하고 동일 평면에서도 안정적이지 않아 우리의 정확성 요구를 충족하지 못한다.

같은 이유로 축 정렬이 아닌 평면 또는 공간 해시에 의존하는 가속 자료구조도 사용할 수 없다. 광선 교차 질의에 원치 않는 근사 오차를 만드는 산술이 필요하기 때문이다. 구현에서는 일반 옥트리를 사용하고 광선 바운딩 박스와 각 옥턴트의 교차를 검사한다. 효율을 위해 분할 단계에서 삼각형 교차 검출에 사용한 것과 같은 가속 구조를 사용한다.

옥트리와 광선이 모두 축 정렬이므로 두 이점이 있다. 광선의 바운딩 박스가 광선 자체로 딱 맞고, 옥턴트 교차가 부동소수점 비교 4번만 필요한 2D 검사로 줄어든다. 광선과 교차한 각 리프 옥턴트의 모든 삼각형을 검사한다. 5.1절처럼 축 정렬 성질을 이용해 광선-삼각형 교차를 2D 삼각형 내 점 질의로 바꾼다. 이 단순한 정식화 덕분에 정확 교차 루틴이 매우 빨라진다.

분류는 광선과 각 입력 메시 사이의 첫 교차 분석에만 의존하므로 교차점을 정렬해야 한다. 먼저 [Attene 2020]의 선-평면 교차(LPI) 점으로 암시적으로 표현한다. 이어 [Cherchi 외 2020]의 정확 비교자로 출발점에서 가까운 순서대로 정렬한다.

5.3. 분류

광선 r과 메시 M의 첫 교차에는 여러 형태가 있다. 가장 단순한 경우는 삼각형 내부의 점에서 표면을 통과하는 것이다(그림 6 왼쪽). 이때 삼각형 정점 순서인 국소 와인딩에 담긴 방향을 분석하면 내부→외부인지 반대인지 알 수 있고 그림 5처럼 정확히 검사할 수 있다.

모서리나 정점에서 교차하면 어렵다. 광선이 표면을 가로지르지 않고 접선 방향으로 지날 수 있고, 실제로 관통하더라도 검사할 삼각형 선택이 모호하기 때문이다. 그림 6은 그중 4가지이며, 접하는 광선이 삼각형과 동일 평면인 경우 등도 있다.

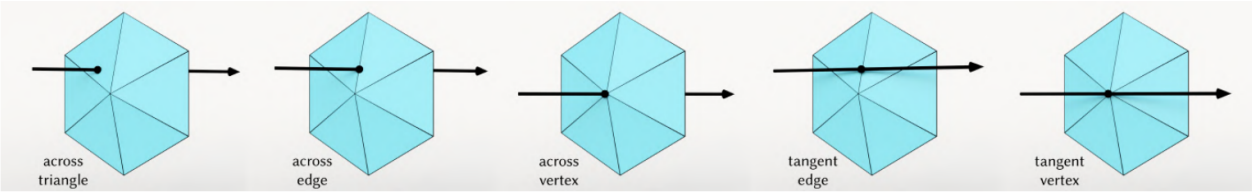


그림 6. 광선과 삼각형 메시 사이의 다섯 교차 사례. 삼각형 내부점에서 표면을 통과하면(맨 왼쪽) 그림 5의 검사로 패치와 표면의 내부·외부 관계를 신뢰성 있게 판단할 수 있다. 정점·모서리에 닿으면 모호해진다. 우리 방법은 광선의 수치 교란으로 항상 맨 왼쪽 유형으로 바꾼다. 이 그림은 모든 병적 사례를 포함하지 않으며, 광선이 동일 평면의 삼각형에 접할 수도 있다.

모든 유형을 구분하는 정확한 기하 검사를 할 수도 있지만, 병적인 사례의 검출·처리 비용을 더 단순하고 효과적인 전략으로 대체한다. 광선이 패치 삼각형 내부점에서 교차하지 않을 때마다 출발점은 유지하고 p_∞ 좌표를 ε 만큼 교란하여, 삼각형 내부점에서 관통할 때까지 반복한다. std::nextafter로 현재 값 다음에 표현 가능한 부동소수점을 사용한다. 교란한 광선을 다시 검사하고 유효한 교차점을 찾을 때까지 반복한다.

교란 후 광선은 더 이상 축 정렬이 아니므로 2D로 줄이지 못하고 3D에서 검사해야 한다. 전체 검사는 두 광선 끝점과 각 삼각형 모서리의 두 끝점을 사용해 만든 사면체 3개의 부호를 확인한다. 모두 엄격히 양수 또는 모두 엄격히 음수면 삼각형 내부에서 교차한다. 추가 비용은 작다. 2D는 orient2D 3회, 3D는 orient3D 3회이다.

실제 형상에서 정점·모서리 교차는 매우 드물다. 대규모 벤치마크의 약 3800 Boolean과 8만 개 이상 광선은 모두 삼각형 내부에서 교차해 수치 교란을 한 번도 수행하지 않았다. 이 코드는 메시와 X축으로 이동한 복사본 사이의 인공 Boolean 사례로만 검증할 수 있었다. 이때 한 메시 패치에서 나온 모든 광선이 다른 메시 정점을 지났다. 교란 한 번만으로 항상 모호함을 해소하고 교차점을 삼각형 내부로 옮길 수 있었다.

6. 논의

파이프라인은 C++로 구현했다. 정확한 광선 투사와 교차 검출에는 CinoLib[Livesu 2019] 자료구조, 강건한 교차점 질의에는 Indirect Predicates[Attene 2020]를 사용한다. 최대 효율과 병렬 성능을 위해 Google Abseil의 빠른 컨테이너와 Intel TBB도 사용한다.

대화형 응용, 대규모 데이터 일괄 처리, 수백만 삼각형의 거대 메시, 수백 입력 메시의 다항 Boolean을 철저히 시험했다. 비교 기준은 공개된 완전한 정확 Boolean 파이프라인 중 가장 최근 방법인 [Zhou 외 2016]이다. 원 구현 공개 후 최근까지 여러 개선이 있었다. 별도 언급이 없으면 libigl[Jacobson 외 2018]에서 제공하는 최신 구현을 기준으로 한다.

[Cherchi 외 2020]과도 비교하지만, 내부·외부 필터링에 비싼 중간 사면체화가 필요하므로 첫 단계만 비교한다. [Bernstein 2013; Levy 2022] 같은 강건하지 않은 대안은 위상 정확성을 보장하지 못하고 그림 3처럼 실패할 수 있어 제외한다.

아래에서 상세히 보이듯 우리 알고리즘은 모든 실험에서 기존 기술보다 최소 약 10배 우수했고, 실시간 응용에서 상호작용 프레임률을 유지할 수 있는 유일한 기존 정확 방법임을 보였다.

6.1. 대화형 응용

스크립트 애니메이션이 진행되는 단순 작업과, 사용자 행동에 따라 장면의 모든 물체가 시간에 따라 변하는 완전한 동적 작업을 모두 고려했다. 모든 대화형 실험은 성능 코어 8개, RAM 32GB의 M1 Pro MacBook 노트북에서 수행했다. 애니메이션의 부드러움을 판단하도록 화면 녹화를 제출 자료에 첨부했다.

Boolean은 단순하게 적용한다. 각 프레임을 독립적으로 계산하고 이전 프레임의 캐시 데이터를 다음으로 전달하지 않는다. 추가 가속을 위한 이 개선은 향후 과제로 남긴다.

회전 데모. 두 물체가 상대적으로 회전하는 동안 알고리즘이 둘의 Boolean을 계산한다(그림 7). 사용자는 키보드로 합집합·교집합·차집합을 선택한다. 물체 하나의 강체 움직임을 제외하면 둘 다 정적이므로 가장 쉬운 대화형 상황이다. 효율을 위해 렌더링과 Boolean을 별도 스레드에서 실행하고 이중 버퍼링으로 동기화한다.

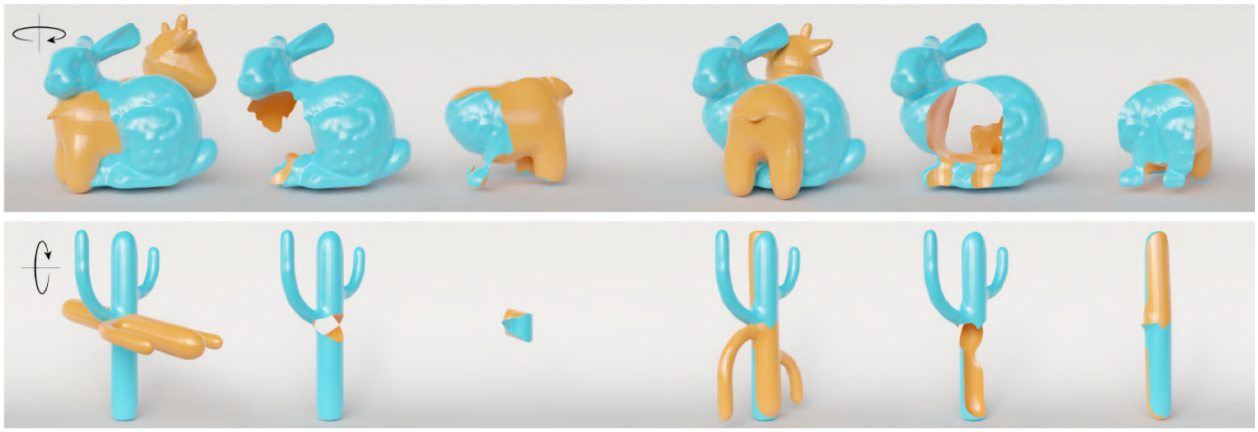


그림 7. 대화형 회전 데모의 두 예. 한 메시가 다른 메시 위에서 회전하는 동안 시스템이 Boolean 연산자를 실시간 실행한다.

삼각형 2.5만~20만 개의 점점 큰 장면을 시험했다. 경험상 형상들이 교차하는 방식은 실행 시간에 상당한 영향을 준다. 예를 들어 두 형상이 거의 완벽히 정렬되면 Boolean이 더 복잡해지고, 그렇지 않으면 점차 단순해진다. 편향을 줄이기 위해 같은 두 모델을 Graphite[2022]로 여러 해상도에 재메시했다.

표 1에 장면별 시간을 제시한다. [Zhou 외 2016] 같은 기존 정확 파이프라인은 가장 거친 해상도부터 대화형 사용에 맞지 않으며 5만 삼각형에서도 약 1~2FPS이다. 비교하면 우리 방법은 100만 삼각형에서 비슷한 FPS로 작동한다. 첫 단계에 [Cherchi 외 2020]을 사용하면 가장 낮은 해상도는 대화형 사용이 가능하지만 2.5만~3만 개 이상으로 확장하기에는 여전히 느리다.

우리 소프트웨어는 약 10만~12만 삼각형까지 대화형으로 작동하고 더 큰 장면에서는 조금 지연되기 시작한다. 부드러움에 대해서는 첨부 녹화를 참조한다.

표 1. 대화형 회전 데모 성능 (초)

총 삼각형 수	libigl 최소/최대/평균	FA 최소/최대/평균	제안 방법 최소/최대/평균
25K	0.22/0.78/0.39	0.03/0.12/0.07	0.01/0.04/0.02
50K	0.36/1.58/0.50	0.12/0.20/0.13	0.02/0.06/0.04
100K	0.69/3.21/0.77	0.23/0.33/0.25	0.03/0.11/0.07
150K	1.03/4.86/1.13	0.34/0.45/0.36	0.05/0.14/0.10
200K	1.37/6.48/1.50	0.46/0.62/0.50	0.06/0.19/0.14

각 장면의 크기는 누적 삼각형 수이며, 연속 3분 실행에서 프레임당 최소·최대·평균 시간을 기록했다. libigl은 [Zhou 외 2016]의 최신 구현이다. FA는 [Cherchi 외 2020]의 원 배열 구현에 우리의 광선 기반 내부·외부 분류를 결합한 하이브리드 파이프라인이다. 모든 시간의 단위는 초이다.

ARAP 변형. 두 삼각형 메시를 ARAP[Sorkine과 Alexa 2007]로 대화형 변형한다. 먼저 마우스로 두 형상에 임의 수의 핸들을 지정한다. 이후 행렬을 분해하고 대화형 세션을 시작한다. 사용자는 어느 형상의 핸들이든 선택해 공간에서 자유롭게 움직인다. 프로그램은 ARAP로 입력 메시를 갱신하고 즉시 둘의 Boolean을 수행한다 (그림 1). 연산자도 실시간으로 변경할 수 있다.

이 데모는 모든 입력이 상호작용 중 상당한 동적 변화를 겪고 ARAP 비용이 Boolean과 대략 같기 때문에 이전보다 어렵다. 최대 효율을 위해 같은 작업 훔치기 스케줄러에서 ARAP와 Boolean 단계를 병렬 실행하고, 이중 버퍼링으로 단계 내 연산을 묶어 지연을 최대한 숨긴다.

모든 실험에서 ARAP를 4회 반복했다. 보통 시각적으로 만족스러운 변형에 충분하다. 첨부 영상에서 10만 삼각형까지는 상당히 부드럽고 15만·20만 장면에서는 점차 지연된다. 자세한 실행 시간은 표 2에 있다.

표 2. 대화형 변형 데모 성능 (초)

총 삼각형 수	최소	최대	평균
25K	0.028	0.055	0.032
50K	0.043	0.079	0.054
100K	0.085	0.119	0.095
150K	0.133	0.164	0.140
200K	0.188	0.215	0.195

ARAP 4회 반복과 제안 Boolean 알고리즘을 수행하는 누적 시간이다.

ARAP는 보장을 제공하지 않는다. 극단적인 핸들 이동은 자기 교차를 만들어 입력 조건을 위반하고 내부·외부 분류를 해칠 수 있다. [Zhang 외 2021]로 회전 행렬을 계산할 때 이런 결함이 가끔 생겼다. Eigen SVD[2010]로 회전 행렬을 계산하자 모든 문제가 해결되었지만 알고리즘은 조금 느려졌다. [Zhang 외 2021]

의 국소 단계가 1.5배 더 빨랐다.

6.2. 대규모 벤치마크

Thingi10K[Zhou와 Jacobson 2016]로 대규모 벤치마크를 수행해 [Zhou 외 2016]과 성능을 비교했다. 저자들이 최근 공개한 상당히 병렬화된 구현을 사용했다. 두 방법 모두 수밀한 다양체 입력을 요구하므로 [Hu 외 2018] 저자들이 공개한 데이터베이스 버전에서 깨끗한 메시 7628개를 추출했다.

이를 각각 3814개인 두 집단으로 나누고 무작위로 짝지어 Boolean을 수행했다. 실제 교차하도록 바운딩 박스를 정규화하고 원점에 중심을 맞췄다. 모든 방법이 정확하므로 출력 위상이 완전히 같음을 보장한다. 이를 이용해 각 출력 메시의 연결 성분 수와 오일러 지표가 같은지 확인해 알고리즘을 검증했다.

실험 장비는 1.2GHz Intel i9, 12코어, RAM 128GB 시스템이다. [Zhou 외 2016]은 3814 Boolean을 모두 처리하는 데 28.3분, 우리 방법은 4.5분 걸렸고 100%의 경우에서 더 빨랐다. 최근 libigl 개선은 원 논문의 구현보다 크게 빨라져, 원래 80분에서 현재 28.3분으로 줄었다.

두 방법 모두 분할에 대부분의 시간을 썼다. [Zhou 외 2016] 22.5분, 우리 방법 4분으로 5.5배 차이다. Boolean 부분은 기존 5.8분, 우리 0.47분으로 12.2배였다. 광선 투사 기반 내부·외부 분류는 최고 101배, 최저 2.38배, 평균 11.34배 빠르다.

6.3. 거대 메시 처리

Thingi10K는 대부분 중간·작은·아주 작은 메시이므로, 수백만 삼각형의 고해상도 메시를 다루는 소규모 실험으로 보완한다. 이런 큰 메시는 대화형 사용에는 부적합하지만 여러 산업의 제작 과정에서 흔하므로 실용적으로 중요하다.



그림 8. 표 3 실험에 사용한 대형 메시 데이터 집합.

그림 8의 고해상도 메시 8개는 각각 삼각형 130만~1440만 개이다. 가능한 모든 메시 쌍에 Boolean을 수행하고 우리 도구, [Zhou 외 2016], 그리고 분할 단계만의 [Cherchi 외 2020] 시간을 측정했다.

표 3처럼 모두 매우 큰 데이터에 잘 확장되고 대체로 실행 시간 비율이 안정적이다. 교차 해소에서는 우리 방법이 [Zhou 외 2016]보다 일관되게 약 10배, [Cherchi 외 2020]보다 평균 5배 빠르다. 가장 큰 변동은 Boolean 부분으로, [Zhou 외 2016] 대비 가속은 약 80배(70% 사례) 또는 약 6배(나머지 사례)이다. 전체 파이프라인 평균 가속은 약 25배이다.

대부분의 실험에서 [Zhou 외 2016]의 병목은 두 번째 Boolean 단계로 실행 시간의 대부분을 차지했다. Thingi10K의 작은 모델에서는 이 부분이 교차 해소보다 거의 무시할 만해 이런 현상이 없었다. 우리는 기존 위상 전파가 매우 큰 메시로 잘 확장되지 않는 반면, 광선 투사 방법은 모든 해상도에서 효율적이라고 추정한다. 평균 53배 빠르며 어떤 실험에서도 계산 시간이 교차 해소 비용을 넘지 않았다.

표 3. 대형 메시의 모든 조합에 대한 성능 비교

표의 가독성을 위해 원래 표를 3a~3c로 나누었다. 입력 두 물체의 누적 크기는 백만 삼각형(M), 시간은 초이다. libigl은 CGAL 5.4 지연 유리수 커널의 개선된 스레드 안전성을 활용하는, 잘 병렬화된 [Zhou 외 2016]의 최상 성능 구현이다. FA는 [Cherchi 외 2020] Fast Arrangement의 저자 기준 구현이다. inters는 교차 해소, bool은 Boolean 부분, tot는 전체이다. 모든 방법이 어느 연산자에서나 비슷한 성능을 보여 합집합만 시험했다. ×는 제안 방법의 가속 배수이다. 평균적으로 전체는 libigl보다 거의 25배, 교차 해소는 FA보다 5배 이상 빠르다. 원 표는 단계별 최고·최저 가속을 파랑·빨강 굵은 글씨로 표시했다. 장비는 성능 코어 8개·RAM 32GB의 M1 Pro MacBook이다.

표 3a. 입력 규모 및 libigl 실행 시간

입력 물체 1	입력 물체 2	크기(M)	교차 해소	Boolean	전체
dragon3	lucy	21.6M	142.81	46.86	189.67
lucy	neptune	18.4M	131.33	42.19	173.52
ganesha	lucy	18.7M	118.19	427.02	545.21
lucy	dragon2	16.6M	117.48	412.19	529.68
horse	lucy	16.6M	116.03	35.83	151.87
lucy	raptor	16.4M	110.12	384.48	494.60
dragon1	lucy	15.7M	110.27	30.12	140.40
dragon3	ganesha	11.5M	65.28	242.44	307.72
dragon3	neptune	11.2M	66.66	20.78	87.44
dragon3	horse	9.4M	56.25	15.03	71.28
dragon3	dragon2	9.4M	54.84	196.39	251.23
dragon3	raptor	9.2M	50.31	193.76	244.07
ganesha	neptune	8.3M	47.39	174.23	221.62
dragon1	dragon3	8.5M	51.05	12.95	64.00
neptune	dragon2	6.2M	44.53	139.26	183.78
ganesha	dragon2	6.5M	35.44	134.41	169.85
ganesha	horse	6.5M	35.22	130.04	165.26
neptune	raptor	6.0M	37.20	138.95	176.15
horse	neptune	6.2M	38.65	11.38	50.03
dragon1	ganesha	5.6M	32.93	119.43	152.35
ganesha	raptor	6.3M	29.68	125.36	155.04
dragon1	neptune	5.3M	32.91	8.99	41.90
horse	dragon2	9.4M	27.28	88.21	115.49
raptor	dragon2	4.2M	25.96	87.85	113.80
horse	raptor	4.2M	24.25	80.78	105.03
dragon1	dragon2	3.5M	22.22	69.96	92.17
dragon1	horse	3.5M	22.06	5.27	27.33
dragon1	raptor	3.3M	16.92	63.55	80.47

표 3b. FA 및 제안 방법 실행 시간

입력 쌍	FA 교차 해소	제안 교차 해소	제안 Boolean	제안 전체
dragon3 / lucy	97.49	12.90	6.61	19.50
lucy / neptune	79.87	12.00	6.13	18.13
ganesha / lucy	84.02	11.30	6.04	17.34
lucy / dragon2	80.10	10.63	5.38	16.01
horse / lucy	74.20	10.41	5.27	15.68
lucy / raptor	68.10	10.01	5.27	15.28
dragon1 / lucy	69.26	9.60	4.90	14.49
dragon3 / ganesha	31.69	6.25	3.09	9.34
dragon3 / neptune	38.56	6.15	2.98	9.12
dragon3 / horse	26.62	5.19	2.31	7.49
dragon3 / dragon2	30.46	5.00	2.38	7.38
dragon3 / raptor	21.82	4.70	2.27	6.97
ganesha / neptune	25.14	4.49	2.27	6.75
dragon1 / dragon3	27.82	4.67	2.06	6.72
neptune / dragon2	32.65	4.02	2.04	6.06
ganesha / dragon2	14.90	3.62	1.72	5.34
ganesha / horse	13.46	3.48	1.67	5.15
neptune / raptor	20.62	3.37	1.70	5.08
horse / neptune	22.07	3.43	1.63	5.06
dragon1 / ganesha	11.59	3.47	1.39	4.86
ganesha / raptor	11.87	3.18	1.61	4.79
dragon1 / neptune	18.55	2.76	1.40	4.15
horse / dragon2	10.44	2.36	1.11	3.47
raptor / dragon2	10.54	2.24	1.05	3.29
horse / raptor	8.34	2.08	0.98	3.07
dragon1 / dragon2	8.33	2.01	0.86	2.88
dragon1 / horse	7.75	1.90	0.83	2.73
dragon1 / raptor	6.37	1.54	0.75	2.29

표 3c. 제안 방법의 가속 배수

입력 쌍	libigl 대비 교차 해소	libigl 대비 Boolean	libigl 대비 전체	FA 대비 교차 해소
dragon3 / lucy	11.07×	7.09×	9.73×	7.56×
lucy / neptune	10.95×	6.88×	9.57×	6.66×
ganesha / lucy	10.46×	70.76×	31.45×	7.43×
lucy / dragon2	11.05×	76.67×	33.08×	7.53×
horse / lucy	11.14×	6.80×	9.68×	7.12×
lucy / raptor	11.00×	72.94×	32.37×	6.80×
dragon1 / lucy	11.49×	6.15×	9.69×	7.22×
dragon3 / ganesha	10.44×	78.38×	32.93×	5.07×
dragon3 / neptune	10.84×	6.98×	9.58×	6.27×
dragon3 / horse	10.85×	6.51×	9.51×	5.13×
dragon3 / dragon2	10.98×	82.38×	34.04×	6.10×
dragon3 / raptor	10.71×	85.28×	35.02×	4.65×
ganesha / neptune	10.57×	76.82×	32.82×	5.61×
dragon1 / dragon3	10.94×	6.29×	9.52×	5.96×
neptune / dragon2	11.08×	68.26×	30.33×	8.12×
ganesha / dragon2	9.80×	78.15×	31.83×	4.12×
ganesha / horse	10.12×	77.82×	32.08×	3.87×
neptune / raptor	11.03×	81.64×	34.71×	6.11×
horse / neptune	11.28×	6.99×	9.90×	6.44×
dragon1 / ganesha	9.50×	85.79×	31.37×	3.35×
ganesha / raptor	9.35×	77.82×	32.39×	3.74×
dragon1 / neptune	11.93×	6.44×	10.09×	6.72×
horse / dragon2	11.56×	79.40×	33.26×	4.42×
raptor / dragon2	11.57×	84.06×	34.60×	4.70×
horse / raptor	11.63×	82.18×	34.24×	4.00×
dragon1 / dragon2	11.04×	80.97×	32.04×	4.14×
dragon1 / horse	11.61×	6.35×	10.01×	4.08×
dragon1 / raptor	10.97×	84.96×	35.13×	4.13×

6.4. 다항 Boolean 연산

메시 크기뿐 아니라 입력 물체 수도 성능에 영향을 준다. 특히 각 입력 형상에 내부·외부 관계를 판정해야 하는 두 번째 단계에서 그렇다. 입력 메시 수에 대한 확장성을 평가하기 위해 기본 메시 A에서 서로 교차하지 않는 작은 장식 요소 B1∪B2∪⋯∪Bn을 뺀다. 장식은 Poisson 샘플링[Corsini 외 2012]으로 표면에 배치한다.

입력 수의 영향만 분리하기 위해 두 번 수행했다. 먼저 A {B1∪B2∪⋯∪Bn}에 n+1개 메시를 따로 입력하여 다항 Boolean으로 계산한다. 다음으로 장식을 하나의 메시 B로 합치고 일반적인 두 메시 연산 A B를 수행한다. 장식끼리 간섭하지 않아 배열과 결과는 동일하다. 그림 9는 두 결과이다.



그림 9. 수백 형상을 포함하는 다항 Boolean에 최적으로 확장된다. 장식 요소를 각각 따로 입력하는 방식과, 모든 장식을 하나의 메시로 합치는 방식으로 실행했다. 두 경우 시간이 같아 입력 형상 수 증가로 계산 오버헤드가 생기지 않았다.

첫 실험에서는 Fertility 조각상(6만 삼각형)에서 작은 구 700개(각 320개, 총 22만4000 삼각형)를 파냈다. 두 방식 모두 같은 결과를 1.1초에 정확히 계산하여, 입력 수 증가에 따른 측정 가능한 오버헤드가 없었다.

두 번째 실험에서는 꽃병(70만 삼각형)에서 10종류의 작은 돌 500개를 뺐다. 각 돌은 5000~1만6000 삼각형, 전체는 470만 삼각형이며 표면에 무작위 방향과 위치로 배치했다. 다항·두 메시 Boolean 시간은 각각 7.49초와 7.59초로 거의 같았다.

이는 입력 형상 수와 관계없이 표면 패치마다 광선 하나를 발사하는 내부·외부 표지 덕분이다. 위상적 전파 기반 기존 방법은 같은 성질이 없으며 입력 수가 커지면 불필요한 오버헤드를 만든다. 첫 실험의 [Zhou 외 2016]은 두 실행 사이에서 5배 이상 느려져 각각 6.8초와 38초였다. 두 번째는 61.01초와 170.93초로 2.8배였다.

7. 결론

위상적으로 정확한 메시 Boolean 계산을 위한 새로운 파이프라인을 제시했다. 주요 기여는 [Cherchi 외 2020] 배열 알고리즘의 개선과 정확한 광선 투사 기반의 새로운 내부·외부 분류이다. 실험처럼 두 기여는 기존 기술보다 상당히 빠르며 전체적으로 최소 약 10배 가속한다.

덕분에 기본 기하 처리와 실시간 Boolean을 결합하는 대화형 응용을 구현했다. 실제 규모의 메시에서 강건한 Boolean과 대화형 도구를 함께 사용하는 것은 이번이 처음이다. 디지털 아티스트와 콘텐츠 제작자가 도구를 빠르게 활용하길 기대하며, 이를 통해 무엇을 만들지 궁금하다.

7.1. 한계 및 향후 연구

현재 시스템에는 두 한계가 있다. 매우 고해상도인 메시, 예를 들어 20만 삼각형을 넘는 메시에서는 상호작용 가능한 프레임률을 달성하지 못하고, 연속 적용되는 Boolean 연산을 강건하게 수행하지 못한다.

확장성. 대화형 모드의 확장성을 개선하는 가장 쉽게 시도할 방법은 프레임 사이에서 부분 계산을 캐시하는 것이다. 6.1절처럼 현재 알고리즘은 시간적 일관성을 활용하지 않고 매번 Boolean을 처음부터 계산하므로 최적이 아니다. 교차 검출과 광선 투사의 가속 자료구조를 한 번 만들고 매 프레임 최소한만 갱신하면 비용을 크게 줄일 수 있다. 현재 매 프레임 처음부터 계산하는 인접성 데이터도 대부분 캐시할 수 있다.

연속 Boolean. 현재 도구는 각 프레임을 이전 프레임과 독립적으로 생성할 수 있는 CSG 형태의 응용으로 제한된다. 이전 Boolean 결과를 다음 입력으로 사용하는 연속 Boolean은 지원하지 않는다. 기술적으로 연속 적용은 가능하지만 매 프레임 정확 좌표를 부동소수점으로 스냅하여 작은 메시 결함을 만들 수 있어 후속 결과를 보장할 수 없다.

6.1절 말미처럼 ARAP 등 자기 관통 부재를 보장하지 않는 기하 처리와 Boolean을 결합할 때도 비슷한 문제가 생길 수 있다. [Diazzi와 Attene 2021]은 처리 중 입력을 암시적으로 복구해 해결했지만 중간 체적 구성이

필요하여 느려진다.

스냅 반올림 문제의 해결이 궁극적인 해법이나, 앞서 설명했듯 매우 어렵다. 스냅을 생략하고 암시적 점을 단순히 전달할 수도 없다. 반복적인 암시적 점 합성은 곧 복잡한 다항식을 만들어 거의 모든 경우에 산술 필터링이 실패하고 심각하게 느려지며 결국 메모리가 부족해진다.

한편 Boolean 출력은 항상 입력의 참값 기본 요소의 부분집합이므로 이러한 반복 합성이 연속 연산에 반드시 필요한 것은 아니라고 본다. 그러나 2차 수준의 기하 구성을 피하도록 암시적 교차점 정의를 갱신하는 적절한 메커니즘은 아직 없다. 평면 기반 표현에서는 비슷한 생각을 [Nehring-Wirxel 외 2021]이 탐구했다. Indirect Predicates[Attene 2020]의 연속 연산을 실현하는 것은 흥미로운 향후 연구 방향이다.

감사의 글

M. Livesu와 M. Attene의 연구는 EU ERC Advanced CHANGE(694515), DIGITbrain/ProMED(952071)의 지원을 일부 받았다. G. Cherchi는 PON R&I 2014-2020 AIM1895943-1의 연구 지원에 감사한다. F. Pellacini는 MIUR의 Dipartimenti di Eccellenza 지원을 일부 받았다.

참고문헌

서지 정보는 원문 표기를 유지했다.

Thomas Alderighi, Luigi Malomo, Daniela Giorgi, Bernd Bickel, Paolo Cignoni, and Nico Pietroni. 2019. Volume-aware design of composite molds. ACM Transactions on Graphics (2019).

Thomas Alderighi, Luigi Malomo, Daniela Giorgi, Nico Pietroni, Bernd Bickel, and Paolo Cignoni. 2018. Metamolds: computational design of silicone molds. ACM Transactions on Graphics (TOG) 37, 4 (2018), 1–13.

Marco Attene. 2014. Direct repair of self-intersecting meshes. Graphical Models 76, 6 (2014), 658–668.

Marco Attene. 2018. As-exact-as-possible repair of unprintable STL files. Rapid Proto- typing Journal (2018).

Marco Attene. 2020. Indirect predicates for geometric constructions. Computer-Aided Design 126 (2020), 102856.

Gavin Barill, Neil G Dickson, Ryan Schmidt, David IW Levin, and Alec Jacobson. 2018. Fast winding numbers for soups and clouds. ACM Transactions on Graphics (TOG) 37, 4 (2018), 1–12.

Audrey Baxter and Lorelei Wrigh. 2019. Ray-Traced Constructive Solid Geometry. (2019).

Gilbert Bernstein. 2013. Cork Boolean Library. https://github.com/gilbo/cork.

Gilbert Bernstein and Don Fussell. 2009. Fast, exact, linear booleans. In Computer Graphics Forum, Vol. 28. Wiley Online Library, 1269–1278.

Blender Doc. 2022. Blender. https://docs.blender.org/manual/en/latest/modeling/modifiers/generate/booleans.html

Marcel Campen and Leif Kobbelt. 2010a. Exact and robust (self-) intersections for polygonal meshes. In Computer Graphics Forum, Vol. 29. Wiley Online Library, 397–406.

Marcel Campen and Leif Kobbelt. 2010b. Polygonal boundary evaluation of minkowski sums and swept volumes. In Computer Graphics Forum, Vol. 29. Wiley Online Library, 1613–1622.

Shouxin Chen, Ming Chen, and Shenglian Lu. 2022. A Real Time Visual Boolean Operation on Triangular Mesh Models. (2022).

Gianmarco Cherchi, Marco Livesu, Riccardo Scateni, and Marco Attene. 2020. Fast and Robust Mesh Arrangements Using Floating-Point Arithmetic. ACM Trans. Graph. 39, 6, Article 250 (Nov. 2020), 16 pages. https://doi.org/10.1145/3414685.3417818

Massimiliano Corsini, Paolo Cignoni, and Roberto Scopigno. 2012. Efficient and flexible sampling with blue noise properties of triangular meshes. IEEE transactions on visualization and computer graphics 18, 6 (2012), 914–924.

Chengkai Dai, Charlie CL Wang, Chenming Wu, Sylvain Lefebvre, Guoxin Fang, and Yong-Jin Liu. 2018. Support-free volume printing by multi-axis motion. ACM Transactions on Graphics (TOG) 37, 4 (2018), 1–14.

Salles Viana Gomes de Magalhães, W Randolph Franklin, and Marcus Vinícius Alvim Andrade. 2020. An Efficient and Exact Parallel Algorithm for Intersecting Large 3-D Triangular Meshes Using Arithmetic Filters. Computer-Aided Design 120 (2020), 102801.

Olivier Devillers, Sylvain Lazard, and William J. Lenhart. 2018. 3D Snap Rounding. In 34th International Symposium on Computational Geometry (SoCG 2018) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 99), Bettina Speckmann and Csaba D. Tóth (Eds.). Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany, 30:1–30:14. https://doi.org/10.4230/LIPIcs.SoCG.2018.30

O. Devillers and F. P. Preparata. 1998. A Probabilistic Analysis of the Power of Arithmetic Filters. Discrete & Computational Geometry 20, 4 (01 Dec 1998), 523–547. https: //doi.org/10.1007/PL00009400

Lorenzo Diazzi and Marco Attene. 2021. Convex polyhedral meshing for robust solid modeling. ACM Transactions on Graphics (TOG) 40, 6 (2021), 1–16.

David Eberly. 2008. Triangulation by ear clipping. Geometric Tools (2008), 2002–2005.

Filippo A Fanni, Gianmarco Cherchi, Alessandro Muntoni, Alessandro Tola, and Ric- cardo Scateni. 2018. Fabrication oriented shape decomposition using polycube mapping. Computers & Graphics 77 (2018), 183–193.

Steven Fortune. 1999. Vertex-rounding a three-dimensional polyhedral subdivision. Discrete & Computational Geometry 22, 4 (1999), 593–618.

Akash Garg, Alec Jacobson, and Eitan Grinspun. 2016. Computational design of recon- figurables. ACM Trans. Graph. 35, 4 (2016), 90–1.

Gaël Guennebaud, Benoît Jacob, et al. 2010. Eigen v3. http://eigen.tuxfamily.org.

Si Hang. 2015. TetGen, a Delaunay-based quality tetrahedral mesh generator. ACM Trans. Math. Softw 41, 2 (2015), 11.

Yixin Hu, Teseo Schneider, Bolun Wang, Denis Zorin, and Daniele Panozzo. 2020. Fast tetrahedral meshing in the wild. ACM Transactions on Graphics (TOG) 39, 4 (2020), 117–1.

Yixin Hu, Qingnan Zhou, Xifeng Gao, Alec Jacobson, Denis Zorin, and Daniele Panozzo. 2018. Tetrahedral Meshing in the Wild. ACM Trans. Graph. 37, 4, Article 60 (jul 2018), 14 pages. https://doi.org/10.1145/3197517.3201353

Alec Jacobson. 2017. Generalized matryoshka: Computational design of nesting objects. In Computer Graphics Forum, Vol. 36. Wiley Online Library, 27–35.

Alec Jacobson, Ladislav Kavan, and Olga Sorkine-Hornung. 2013. Robust inside-outside segmentation using generalized winding numbers. ACM Transactions on Graphics (TOG) 32, 4 (2013), 1–12.

Alec Jacobson, Daniele Panozzo, et al. 2018. libigl: A simple C++ geometry processing library. https://libigl.github.io/.

Bruno Lévy. 2016. Robustness and efficiency of geometric programs: The Predicate Construction Kit (PCK). Computer-Aided Design 72 (2016), 3–12.

Bruno Levy. 2022. Graphite. https://github.com/BrunoLevy/GraphiteThree.

Marco Livesu. 2019. cinolib: a generic programming header only C++ library for processing polygonal and polyhedral meshes. Transactions on Compu- tational Science XXXIV (2019). https://doi.org/10.1007/978-3-662-59958-7_4 https://github.com/mlivesu/cinolib/.

Marco Livesu, Gianmarco Cherchi, Riccardo Scateni, and Marco Attene. 2021. De- terministic Linear Time Constrained Triangulation using Simplified Earcut. IEEE Transactions on Visualization and Computer Graphics (2021).

Salles VG Magalhães, W Randolph Franklin, and Marcus VA Andrade. 2017. Fast exact parallel 3D mesh intersection algorithm using only orientation predicates. In Proceedings of the 25th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems. 1–10.

Ahmed H Mahmoud, Serban D Porumbescu, and John D Owens. 2021. RXMesh: a GPU mesh data structure. ACM Transactions on Graphics (TOG) 40, 4 (2021), 1–16.

Célestin Marot, Jeanne Pellerin, and Jean-François Remacle. 2019. One machine, one minute, three billion tetrahedra. Internat. J. Numer. Methods Engrg. 117, 9 (2019), 967–990.

Maya Doc. 2022. AutoDesk. https://knowledge.autodesk.com/support/maya/learn-explore/caas/CloudHelp/cloudhelp/2020/ENU/Maya-Modeling/files/GUID-302821C5-343C-4F2B-8228-C5333896B207-htm.html

Gang Mei and John C Tipper. 2013. Simple and robust boolean operations for triangu- lated surfaces. arXiv preprint arXiv:1308.4434 (2013).

Victor Milenkovic and Elisha Sacks. 2019. Geometric rounding and feature separation in meshes. Computer-Aided Design 108 (2019), 12–18.

Tomas Möller and Ben Trumbore. 1997. Fast, minimum storage ray-triangle intersection. Journal of graphics tools 2, 1 (1997), 21–28.

Alessandro Muntoni, Marco Livesu, Riccardo Scateni, Alla Sheffer, and Daniele Panozzo. 2018. Axis-aligned height-field block decomposition of 3D shapes. ACM Transactions on Graphics (TOG) 37, 5 (2018), 1–15.

Julius Nehring-Wirxel, Philip Trettner, and Leif Kobbelt. 2021. Fast exact booleans for iterated CSG using octree-embedded BSPs. Computer-Aided Design 135 (2021), 103015.

Stefano Nuvoli, Alex Hernandez, Claudio Esperança, Riccardo Scateni, Paolo Cignoni, and Nico Pietroni. 2019. QuadMixer: layout preserving blending of quadrilateral meshes. ACM Transactions on Graphics (TOG) 38, 6 (2019), 1–13.

Alberto Paoluzzi, Vadim Shapiro, Antonio DiCarlo, Francesco Furiani, Giulio Martella, and Giorgio Scorzelli. 2020. Topological computing of arrangements with (co) chains. ACM Transactions on Spatial Algorithms and Systems (TSAS) 7, 1 (2020), 1–29.

Alberto Paoluzzi, Vadim Shapiro, Antonio DiCarlo, Giorgio Scorzelli, and Elia Onofri. 2019. Finite Boolean Algebras for Solid Geometry using Julia’s Sparse Arrays. arXiv preprint arXiv:1910.11848 (2019).

Alexander Pasko, V Adzhiev, R Cartwright, E Fausett, A Ossipov, and V Savchenko. 1999. HyperFun project: A framework for collaborative multidimensional F-rep modeling. In Eurographics/ACM SIGGRAPH Workshop Implicit Surfaces’ 99. 59–69.

Darko Pavić, Marcel Campen, and Leif Kobbelt. 2010. Hybrid booleans. In Computer Graphics Forum, Vol. 29. Wiley Online Library, 75–87.

Sylvain Pion and Andreas Fabri. 2011. A generic lazy evaluation scheme for exact geometric computations. Science of Computer Programming 76, 4 (2011), 307 – 323. https://doi.org/10.1016/j.scico.2010.09.003 Special issue on library-centric software design (LCSD 2006).

Martin Schifko, Bert Jüttler, and Bernhard Kornberger. 2010. Industrial application of exact boolean operations for meshes. In Proceedings of the 26th Spring Conference on Computer Graphics. 165–172.

Silvia Sellán, Noam Aigerman, and Alec Jacobson. 2021. Swept volumes via spacetime numerical continuation. ACM Transactions on Graphics (TOG) 40, 4 (2021), 1–11.

Bin Sheng, Bowen Liu, Ping Li, Hongbo Fu, Lizhuang Ma, and Enhua Wu. 2018. Accel- erated robust Boolean operations based on hybrid representations. Computer Aided Geometric Design 62 (2018), 133–153.

Jonathan Richard Shewchuk. 1996. Triangle: Engineering a 2D quality mesh generator and Delaunay triangulator. In Workshop on applied computational geometry. Springer, 203–222.

Jonathan Richard Shewchuk. 1997. Adaptive precision floating-point arithmetic and fast robust geometric predicates. Discrete & Computational Geometry 18, 3 (1997), 305–363.

Jonathan Richard Shewchuk and Brielin C Brown. 2015. Fast segment insertion and incremental construction of constrained Delaunay triangulations. Computational Geometry 48, 8 (2015), 554–574.

Olga Sorkine and Marc Alexa. 2007. As-rigid-as-possible surface modeling. In Sympo- sium on Geometry processing, Vol. 4. 109–116.

Kokichi Sugihara, Masao Iri, et al. 1989. A solid modelling system free from topological inconsistency. Journal of Information Processing 12, 4 (1989), 380–393.

Michael Tao, Christopher Batty, Eugene Fiume, and David IW Levin. 2019. Mandoline: robust cut-cell generation for arbitrary triangle meshes. ACM Transactions on Graphics (TOG) 38, 6 (2019), 1–17.

Philip Trettner, Julius Nehring-Wirxel, and Leif Kobbelt. 2022. EMBER: exact mesh booleans via efficient & robust local arrangements. ACM Transactions on Graphics (TOG) 41, 4 (2022), 1–15.

Francisca Gil Ureta, Chelsea Tymms, and Denis Zorin. 2016. Interactive modeling of mechanical objects. In Computer Graphics Forum, Vol. 35. Wiley Online Library, 145–155.

Brian Wyvill, Andrew Guy, and Eric Galin. 1999. Extending the csg tree. warping, blending and boolean operations in an implicit surface modeling system. In Computer Graphics Forum, Vol. 18. Wiley Online Library, 149–158.

Songgang Xu and John Keyser. 2013. Technical Note: Fast and Robust Booleans on Polyhedra. 45, 2 (feb 2013), 529–534. https://doi.org/10.1016/j.cad.2012.10.036

Jiaxian Yao, Danny M Kaufman, Yotam Gingold, and Maneesh Agrawala. 2017. In- teractive design and stability analysis of decorative joinery for furniture. ACM Transactions on Graphics (TOG) 36, 2 (2017), 1–16.

Cédric Zanni, Frédéric Claux, and Sylvain Lefebvre. 2018. HCSG: Hashing for real- time CSG modeling. Proceedings of the ACM on Computer Graphics and Interactive Techniques 1, 1 (2018), 1–19.

Jiayi Eris Zhang, Alec Jacobson, and Marc Alexa. 2021. Fast Updates for Least-Squares Rotational Alignment. Computer Graphics Forum (2021).

Qingnan Zhou, Eitan Grinspun, Denis Zorin, and Alec Jacobson. 2016. Mesh arrange- ments for solid geometry. ACM Transactions on Graphics (TOG) 35, 4 (2016), 1–15.

Qingnan Zhou and Alec Jacobson. 2016. Thingi10K: A Dataset of 10,000 3D-Printing Models. https://doi.org/10.48550/ARXIV.1605.04797